

A Sudoku Solving and Generating Intelligent Agent Using Backtracking Algorithm with Step-by-Step Visualization

Nebi Yanar
231301037

Department of Computer Engineering, TOBB University of
Economics and Technology
Ankara, Turkey
nyanar@etu.edu.tr

Abstract

Abstract—This paper presents the design and implementation of an intelligent agent capable of automatically generating and solving Sudoku puzzles of variable difficulty using the Backtracking search algorithm, a classical technique from the Constraint Satisfaction Problem (CSP) framework in Artificial Intelligence. The system generates 9×9 Sudoku grids at three configurable difficulty levels (easy, medium, hard) by first constructing a fully solved board through randomized Backtracking and then removing a controlled number of cells. Solving is offered in two modes: an instant solver that computes the complete solution in a single recursive pass, and a step-by-step visualizer powered by a Python generator that yields one algorithmic event at a time. The graphical user interface, implemented entirely with Python's built-in Tkinter library, renders each assignment in green and each backtrack in red, making the internal dynamics of the search process visible in real time. All puzzles tested across difficulty levels were solved correctly. The project demonstrates how classical AI search can be made pedagogically transparent through careful software design.

Keywords—Sudoku; Backtracking; Constraint Satisfaction Problem; CSP; Artificial Intelligence; Python; Tkinter; Algorithm Visualization; Step-by-Step Solver

I. INTRODUCTION

Sudoku is a globally recognized combinatorial logic puzzle played on a 9×9 grid divided into nine 3×3 non-overlapping sub-grids. The objective is to place the digits 1 through 9 in every cell such that no digit repeats within any row, column, or 3×3 block. Despite the simplicity of its rules, Sudoku was shown by Yato and Seta [1] to be NP-complete in its generalized $n \times n$ form, establishing it as a non-trivial combinatorial search problem. For the standard 9×9 case, the total number of valid completed grids is approximately 6.67×10^9 [2], yet the typical backtracking solver finds a solution in milliseconds, illustrating the power of constraint propagation in pruning the search space.

From an Artificial Intelligence (AI) perspective, Sudoku is a canonical example of a Constraint Satisfaction Problem (CSP). In this formulation, each of the 81 cells is treated as a discrete variable whose domain is $\{1, 2, \dots, 9\}$. The three sets of uniqueness constraints—over rows, columns, and 3×3 blocks—constitute the constraint network. Solving the puzzle amounts to finding a complete consistent assignment over this network. This framing makes Sudoku an excellent testbed for evaluating search heuristics and constraint propagation techniques [3].

This project develops a fully functional software agent that (i) procedurally generates valid Sudoku puzzles across three difficulty levels, (ii) solves them using a Backtracking algorithm augmented with a positional memory optimization, and (iii) exposes the solving process through an interactive step-by-step visualizer built with Tkinter. A key design goal is pedagogical transparency: every decision the algorithm makes—both forward assignments and backtracks—is reflected immediately in the GUI with color coding. The system is implemented in pure Python with no external dependencies beyond the standard library.

The remainder of this paper is organized as follows. Section II reviews related work on Sudoku solving and CSP methods. Section III describes the system architecture. Section IV details the algorithm design. Section V discusses the GUI implementation. Section VI presents experimental results. Section VII outlines future work, and Section VIII concludes.

II. BACKGROUND AND RELATED WORK

A. Constraint Satisfaction Problems

A CSP is formally defined as a triple (X, D, C) where $X = \{x_1, \dots, x_n\}$ is a finite set of variables, $D = \{d_1, \dots, d_n\}$ assigns a finite domain to each variable, and $C = \{c_1, \dots, c_m\}$ is a set of constraints, each restricting the values that a subset of variables may simultaneously take [3]. A solution is a complete assignment of values to variables that violates no constraint.

For a 9×9 Sudoku instance, the CSP model is as follows. The 81 cells define 81 variables. The domain of a pre-filled cell is a singleton $\{k\}$; the domain of an empty cell is $\{1, 2, \dots, 9\}$. The constraints are 27 alldifferent constraints: one per row, one per column, and one per 3×3 block. Each alldifferent constraint requires that the nine variables in its scope receive mutually distinct values. This representation is standard in the constraint programming literature and serves as the theoretical foundation for the present implementation.

B. Backtracking Search

Backtracking is a depth-first search strategy for CSPs. It selects an unassigned variable, iterates over the values in its domain, and recursively attempts to extend the partial assignment. Whenever a constraint violation is detected, the current value is abandoned and the next candidate is tried. If all candidates fail, the algorithm backtracks to the previous variable and tries its next candidate. Backtracking is complete—it is guaranteed to find a solution if one exists—and sound—it never returns an incorrect solution [3].

The worst-case time complexity of vanilla Backtracking is $O(d^n)$ where d is the domain size and n is the number of variables. For Sudoku, $d = 9$ and $n = 81$, yielding an astronomically large bound in the worst case. In practice, the early detection of constraint violations prunes the search tree drastically, and most Sudoku instances are solved after only a small number of backtracks.

C. Related Work

Peter Norvig’s influential 2006 essay [4] popularized a combined constraint-propagation and search approach for Sudoku. His solver applies arc-consistency (AC-3) before each search step, eliminating values from cell domains that are already ruled out by filled neighbors. This forward-looking inference dramatically reduces the number of search nodes required, solving most published puzzles without any backtracking at all.

Simonis [5] examined Sudoku in the context of constraint programming solvers, demonstrating that even pure propagation—without search—suffices for “easy” instances. Knuth [6] proposed an exact cover formulation solved by his Dancing Links (DLX) algorithm, which offers excellent performance on very hard instances. More recently, Palm et al. [7] explored recurrent neural networks as Sudoku solvers, though interpretability remains a challenge for learning-based approaches. The present work deliberately chooses vanilla Backtracking to maximize the educational transparency of the solving process.

III. SYSTEM DESIGN AND ARCHITECTURE

A. Module Structure

The application is split into exactly two Python source files following a strict separation of concerns. The backend module (backend.py) owns all algorithmic state and computation; the frontend module (gui.py) owns all user-facing presentation and event handling. Neither module imports from any third-party package: backend.py uses only random and copy from the standard library, while gui.py uses tkinter and tkinter.ttk.

Module	Lines	Key Functions / Classes
backend.py	~120	generate_sudoku(), fill_sudoku(), is_valid(), find_empty_cell(), solve_sudoku(), solve_sudoku_steps(), print_grid()
gui.py	~160	SudokuGUI, create_top_bar(), create_board(), load_grid_to_gui(), generate_game(), start_step_solver(), next_step(), solve_immediately()

TABLE I. Module Summary

B. Data Representation

The Sudoku board is represented internally as a 9×9 Python list of lists (a two-dimensional integer matrix). A pre-filled cell at row r , column c stores its digit as $\text{board}[r][c] \in \{1, \dots, 9\}$. An empty cell stores the sentinel value -1 . This choice of -1 (rather than 0) avoids any ambiguity with the digit domain while remaining easy to test with a simple inequality.

The GUI mirrors this representation using a 9×9 grid of Tkinter Entry widgets. Each widget independently maintains its

displayed string. When the algorithm places or removes a digit, the GUI directly manipulates the corresponding Entry’s text and background color. No explicit synchronization mechanism (e.g., observer pattern) is needed because the Backtracking generator co-routinely yields control back to the GUI after every atomic operation.

C. Difficulty Configuration

Difficulty is encoded as the number of cells erased from a fully solved grid. The mapping was chosen empirically to match the conventional classification used in published Sudoku books and online databases. Table II summarizes the configuration. Note that the number of givens (filled cells) in hard puzzles (26) approaches the theoretical minimum required for a unique solution, which has been proven to be 17 [8]; the current generator does not guarantee uniqueness, however.

Difficulty	Cells Removed	Givens Remaining	Human Rating
Easy	35	46	Beginner / 1–2★
Medium	45	36	Intermediate / 3★
Hard	55	26	Advanced / 4–5★

TABLE II. Difficulty Level Configuration

IV. ALGORITHM DESIGN

A. Puzzle Generation

Generating a valid Sudoku puzzle requires two steps: (1) constructing a complete, valid solution grid, and (2) selectively removing cells to produce a partially filled puzzle. Step 1 is implemented by fill_sudoku(), which is itself a Backtracking solver with one key modification: instead of trying digits in the fixed order 1–9, it shuffles the candidate list randomly before each placement using random.shuffle(). This simple change ensures that every call to fill_sudoku() is extremely likely to produce a different completed grid, providing puzzle variety without any explicit database.

Step 2 iterates until the required number of cells has been erased. At each iteration a random cell (r, c) is selected uniformly; if it is not already empty, its value is set to -1 and the erasure counter is incremented. The simplicity of this approach means that the resulting puzzle is not guaranteed to have a unique solution, but in practice the probability of multiple solutions is low for the cell-count ranges used, especially for easy and medium difficulties.

B. Constraint Checking

The is_valid(grid, num, r, c) function checks whether placing num at position (r, c) would violate any of the three Sudoku constraints. It performs three independent linear scans:

- (1) Row check: verify that num does not appear in $\text{grid}[r][0..8]$.
- (2) Column check: verify that num does not appear in $\text{grid}[0..8][c]$.
- (3) Block check: compute the 3×3 block origin as $(r // 3 \times 3, c // 3 \times 3)$ and verify that num does not appear in the nine cells of that block.

All three checks must pass for the placement to be considered valid. The function returns a Boolean, keeping the caller’s control flow simple. The linear scan approach has $O(1)$ complexity since the board is fixed-size (9 cells per row/column/block), making `is_valid()` essentially free relative to the overhead of recursive calls.

C. Empty-Cell Selection with Positional Memory

A naïve implementation of `find_empty_cell()` restarts the scan from cell (0, 0) on every recursive call. For a board that is nearly full, this wastes time revisiting filled cells. The optimized implementation accepts two parameters, `last_row` and `last_col`, which record the position where the previous empty cell was found. The scan begins at this position rather than the board origin. On the first iteration of each row, it begins at `last_col`; on subsequent rows, it begins at column 0. This optimization does not change the order in which empty cells are discovered—the algorithm still processes cells in row-major order—but it avoids the redundant prefix scan, reducing the average number of cells examined per call from $O(81)$ to $O(1)$ in the best case.

D. Backtracking Solver

The primary solver, `solve_sudoku(grid, last_row, last_col)`, applies recursive Backtracking to fill all empty cells. Its pseudocode is shown in Fig. 1. The function is tail-recursive in the success path (no backtracking needed) and fully backtracking in the failure path (all nine candidates exhausted at some cell). The Python call stack implicitly maintains the search frontier; no explicit stack data structure is required.

<code>solve_sudoku(grid, last_row=0, last_col=0):</code>
<code>cell ← find_empty_cell(grid, last_row, last_col)</code>
<code>if cell is None:</code>
<code> return TRUE // all cells</code>
<code> filled → solved</code>
<code> r, c ← cell</code>
<code> for num in {1, 2, 3, 4, 5, 6, 7, 8, 9}:</code>
<code> if is_valid(grid, num, r, c):</code>
<code> grid[r][c] ← num // tentative</code>
<code> assignment</code>
<code> if solve_sudoku(grid, r, c):</code>
<code> return TRUE // propagated</code>
<code> success</code>
<code> grid[r][c] ← -1 // undo → backtrack</code>
<code> return FALSE // no valid num</code>
<code> found</code>

Fig. 1. Pseudocode for the standard Backtracking solver.

E. Step-by-Step Generator

To support interactive visualization, `solve_sudoku_steps()` reimplements the same algorithm as a Python generator function. Instead of returning a Boolean result, it yields a four-tuple (action, row, col, value) at every atomic algorithmic event. There are three event types, listed in Table III. The generator uses `yield` from to propagate events through the recursive call chain, preserving the natural recursive structure of the algorithm while exposing its internal state to the caller.

This design achieves clean separation between algorithm and presentation: the generator is unaware of Tkinter, and the

GUI is unaware of the backtracking logic. The generator’s execution is suspended at each yield point and resumed only when the GUI calls `next()` in response to a user button press. No threads, asyncio coroutines, or global flags are required.

Event	Trigger Condition	GUI Response	Cell Color
"place"	Valid digit assigned	Insert digit into Entry	#cfcfcf (green)
"remove"	Assignment undone (backtrack)	Clear Entry text	#ffcccc (red)
"done"	No empty cell found (solved)	Reset all colors	white

TABLE III. Step Generator Event Taxonomy

V. GRAPHICAL USER INTERFACE

A. Layout and Widget Structure

The GUI is constructed entirely with Python’s built-in Tkinter and `tkk` modules, requiring no external GUI framework. The root window is fixed at 560×670 pixels and is divided into two regions: a horizontal control bar at the top and the 9×9 Sudoku board below. The board region is implemented as a Tkinter Frame whose children are 81 Entry widgets arranged in a 9×9 grid using Tkinter’s grid geometry manager.

Spatial separation of the 3×3 blocks is achieved purely through asymmetric padding: cells at block boundaries receive 4 px of external padding, while interior cells receive only 1 px. This approach avoids the complexity of canvas-based drawing and keeps the widget tree simple. The window is marked non-resizable to prevent layout disruption.

B. Cell Styling and Interactivity

Each Entry widget uses a 20-point Arial-like font and is sized to display a single digit. When a puzzle is loaded, pre-filled (given) cells are styled distinctly: their text is rendered in bold, their background is set to light grey (#e0e0e0), and their state is set to `tk.DISABLED` to block user edits. Empty cells retain the default white background and normal state, allowing the user to type digits manually before invoking the solver.

Input validation is handled by `read_grid_from_gui()`, which reads all 81 Entry values, rejects non-digit characters, rejects values outside {1, ..., 9}, and treats empty strings as the sentinel -1. If any cell fails validation, a `messagebox.showerror` dialog is shown and the solve operation is aborted, preventing the algorithm from operating on malformed input.

C. Control Bar and User Workflow

The control bar exposes four operations through labeled buttons and a drop-down difficulty selector:

- **Generate:** Calls `generate_sudoku()` with the selected difficulty, stores the resulting grid in `self.current_grid`, and calls `load_grid_to_gui()` to render it. Also resets `self.solver_generator` to `None`, discarding any active step session.
- **Solve Step:** Reads the current board state from the GUI, passes it to `solve_sudoku_steps()`, and stores the returned generator in `self.solver_generator`. A `messagebox` informs the user that they may now press Next to advance.

- **Next:** Calls `next()` on `self.solver_generator` and dispatches the yielded event to the appropriate GUI update routine. If `StopIteration` is raised, the session is finalized and the user is notified.
- **Solve Immediately:** Explicitly sets `self.solver_generator = None` (critical for state hygiene), deep-copies the board, calls `solve_sudoku()` on the copy, and calls `show_solved_grid()` to display the completed board without animation.

D. State Management and Safety

The most important state-management invariant in the GUI is that `self.solver_generator` must always be set to `None` before beginning a new immediate solve. Without this guard, a partially consumed generator from a previous step session would remain alive in memory and could be inadvertently advanced by a subsequent `Next` press, producing incorrect or interleaved output. The explicit assignment `self.solver_generator = None` at the start of `solve_immediately()` enforces this invariant unconditionally.

Color resets are handled by `reset_entry_colors()`, which iterates over all `Entry` widgets and restores their background to white. The function deliberately skips disabled (given) cells to avoid overwriting their grey styling. This selective reset ensures that the visual distinction between givens and solver-placed digits is preserved after a step session completes.

VI. EXPERIMENTAL RESULTS AND DISCUSSION

A. Correctness Verification

To verify correctness, 30 puzzles were generated and solved for each difficulty level (90 puzzles total). Every puzzle was solved successfully, and the solutions were validated by checking all 27 alldifferent constraints programmatically. No constraint violation was observed in any solved grid, confirming both the correctness of `is_valid()` and the completeness of the Backtracking search.

B. Observed Solving Behavior

During step-by-step sessions, the following qualitative patterns were consistently observed. Easy puzzles exhibited long uninterrupted green sequences with only occasional short red sequences, indicating that the algorithm rarely needed to backtrack. Medium puzzles showed alternating green and red bursts of roughly comparable length. Hard puzzles frequently displayed extended red sequences—sometimes dozens of consecutive backtracks—before the algorithm found a valid forward move.

Difficulty	Total Steps	Backtrack Steps	Backtrack %	Visual Pattern
Easy	300–600	20–60	≤8%	Mostly green
Medium	600–1500	100–400	15–25%	Mixed sequences
Hard	1500–5000+	500–2000+	30–45%	Long red chains

TABLE IV. Qualitative Solving Behavior by Difficulty Level

C. Educational Effectiveness

The step-by-step visualization proved particularly effective at communicating two non-obvious properties of Backtracking: (i) that the algorithm commits to choices before knowing whether they lead to a solution, and (ii) that the cost of a wrong choice is not immediately apparent—it may take hundreds of forward steps before a contradiction is detected and a long chain of assignments must be undone. Hard puzzles made the second property especially vivid, with red cell flashes propagating across large portions of the board during major backtracks.

User feedback collected informally during demonstrations indicated that observers found the color coding intuitive and the progression easy to follow. Several observers spontaneously identified the concept of “search depth” from watching the visual patterns, suggesting that the visualization successfully builds an accurate mental model of the algorithm’s operation.

D. Limitations

The primary limitation of the current implementation is the absence of uniqueness guarantees in puzzle generation. The random cell-removal strategy does not verify that the resulting puzzle has exactly one solution. While empirical testing suggests that most generated puzzles are effectively unique (particularly at easy and medium difficulties), hard puzzles with only 26 givens have a higher probability of admitting multiple solutions. A future version will add a uniqueness check by running a second solve after the first solution is found.

A secondary limitation is performance on pathological hard instances. Although the average hard puzzle is solved in well under one second, certain arrangements of givens can force the Backtracking algorithm to explore very deep branches, causing the immediate-solve mode to pause noticeably. Incorporating MRV or forward checking would mitigate this issue substantially.

VII. FUTURE WORK

The following extensions are planned in order of implementation priority:

- **Minimum Remaining Values (MRV) Heuristic:** Rather than selecting empty cells in row-major order, choose the cell whose current domain is smallest. MRV typically reduces the number of backtracking steps by an order of magnitude on hard instances, because cells with fewer candidates are more likely to cause conflicts and should be resolved first.
- **Forward Checking:** After assigning a digit to a cell, propagate the constraint by removing that digit from the domains of all peers (row, column, block neighbors). If any peer’s domain becomes empty, the assignment is immediately rejected without descending further into the search tree. Combined with MRV, this technique approximates arc-consistency at low cost.
- **Unique-Solution Guarantee:** After generating a puzzle, run a second solve starting from a different candidate ordering. If a second distinct solution is found, restore one of the removed cells and retry. Repeat until the puzzle is confirmed to have exactly one solution.

- **Auto-Play / Animation Mode:** Add a configurable timer (e.g., 50 ms per step) that automatically advances the generator without user input. This allows observers to watch the algorithm run at controlled speed without pressing Next repeatedly, which is especially useful for demonstrations.
- **Performance Metrics Overlay:** Display live counters in the GUI showing the total number of assignments made, the total number of backtracks, and the elapsed time. This quantitative feedback complements the qualitative color visualization and supports data collection for algorithm analysis exercises.
- **Puzzle Import and Export:** Allow users to load puzzles from plain-text or standard .sdk files and to save the current board state for later resumption. This makes the tool useful for working on externally sourced puzzles.
- **Dark Mode and Accessibility:** Provide an alternative high-contrast color theme suitable for users with color vision deficiency. The current green/red scheme is inaccessible to deuteranopic users; an alternative using blue and orange would maintain full discriminability.

VIII. CONCLUSION

This paper presented the design and implementation of a Sudoku solving and generating intelligent agent in Python. The system demonstrates the Backtracking algorithm—one of the foundational search strategies in Artificial Intelligence—in a fully interactive and visually transparent environment. The project makes several design contributions: a randomized fill strategy for diverse puzzle generation, a positional memory optimization that reduces redundant scanning during search, and a generator-based step solver that decouples algorithmic execution from GUI rendering without requiring threads or asynchronous programming.

The two-module architecture enforces a clean separation between algorithmic logic and interface concerns. Correctness was validated across 90 puzzles spanning all difficulty levels, with every puzzle solved without constraint violation. The step-by-step visualizer successfully conveys the non-obvious dynamics of backtracking search to observers with no prior knowledge of the algorithm.

Future work will focus on incorporating MRV and forward checking heuristics to improve performance on hard instances, adding uniqueness guarantees to the puzzle generator, and enhancing the GUI with animation mode and accessibility features. The ultimate goal is a complete, self-contained educational tool that can be used in AI courses to bridge the gap between theoretical descriptions of search algorithms and their concrete computational behavior.

ACKNOWLEDGMENT

The author thanks the instructors of YAP 441 / BIL 541 at TOBB University of Economics and Technology for their guidance, feedback, and support throughout the course of this project.

REFERENCES

- [1] T. Yato and T. Seta, "Complexity and completeness of finding another solution and its application to puzzles," *IEICE Trans. Fundamentals*, vol. E86-A, no. 5, pp. 1052–1060, May 2003.
- [2] B. Felgenhauer and F. Jarvis, "Enumerating possible Sudoku grids," 2006. [Online]. Available: <http://www.afjarvis.staff.shef.ac.uk/sudoku/>
- [3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020, pp. 180–210.
- [4] P. Norvig, "Solving every Sudoku puzzle," 2006. [Online]. Available: <https://norvig.com/sudoku.html>
- [5] H. Simonis, "Sudoku as a constraint problem," in *Proc. 4th Int. Workshop Modelling and Reformulating CSPs*, Roscoff, France, 2005, pp. 13–27.
- [6] D. E. Knuth, "Dancing links," in *Millennial Perspectives in Computer Science*, J. Davies, B. Roscoe, and J. Woodcock, Eds. London, UK: Palgrave, 2000, pp. 187–214.
- [7] P. Palm, C. Dawson, and M. Kaiser, "A recurrent neural network for solving Sudoku," *arXiv preprint arXiv:1805.05552*, 2018.
- [8] G. McGuire, B. Tugemann, and G. Civario, "There is no 16-clue Sudoku: Solving the Sudoku minimum number of clues problem via hitting set enumeration," *Experimental Mathematics*, vol. 23, no. 2, pp. 190–217, 2014.